

Cursor CLI and Local Automation

Module 5 · Day 1 (Hands-On)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Hands-on exercise
Prerequisites	Cursor CLI installed, terminal access, Modules 1-4 completed
Module Goal	Master the Cursor CLI for terminal-based AI workflows and automation

Learning Objectives

By the end of this module, participants will be able to:

- Use the Cursor CLI in interactive mode for real-time AI collaboration
- Run one-shot CLI commands for scripting and CI/CD integration
- Hand off local sessions to Cloud Agents for remote execution
- List, resume, and manage concurrent sessions effectively

Agenda

Lesson	Topic	Time
5.1	Interactive CLI	20 min
5.2	One-Shot CLI	20 min
5.3	Cloud Handoff	18 min
5.4	Listing and Resuming Sessions	20 min

Lesson 5.1

Interactive CLI

Concept · 8 min · Exercise · 12 min

What Is the Cursor CLI?

The Cursor CLI brings AI-powered coding directly to your command line.

- Start AI sessions from your terminal
- Get code assistance without leaving your workflow
- Automate coding tasks with scripts
- Integrate AI into existing CLI tools

Primary command: `agent` (main entry point)

Interactive Mode Commands

Command	Purpose
<code>/model</code>	Switch between AI models interactively
<code>/compress</code>	Summarize conversation, free up context window
<code>/rules</code>	Create and edit rules directly from CLI
<code>/commands</code>	Create and modify custom commands
<code>/mcp enable/disable</code>	Manage MCP servers
<code>/usage</code>	View Cursor usage stats
<code>/about</code>	View environment and CLI configuration
<code>/resume</code>	View and resume previous sessions

Exercise 5.1 – Steps 1-2

Step 1: Start an interactive session

```
agent  
agent "Help me understand the current codebase structure"
```

Step 2: Navigate the session

- Type prompts naturally
- **Shift+Enter** – new line without submitting
- **Enter** – submit prompt
- **Ctrl+D** twice – exit

Exercise 5.1 – Steps 3–5

Step 3: Switch models:

```
/model  
# Or list models outside session:  
agent --list-models
```

Step 4: Ask Mode (read-only):

```
agent --mode=ask "What does this project's main function do?"  
# Or inside session: /ask
```

Step 5: Plan Mode:

```
agent --mode=plan "Add user authentication to this API"
```

Exercise 5.1 – Steps 6–7

Step 6: Configure status line:

```
npx -y cursor-statusline  
# Shows: [model: claude-4.5-sonnet] [~/project] [main] [ctx: 45k/200k]
```

Step 7: Terminal key bindings:

```
agent /setup-terminal
```

Success Criteria: Started session · switched models · used Ask/Plan Mode · configured status line

Lesson 5.2

One-Shot CLI

Concept · 8 min · Exercise · 12 min

One-Shot Command Structure

```
agent "your prompt here"           # Basic one-shot
agent --mode=ask "question about code" # Read-only
agent --model claude-4.5-sonnet "task" # Specific model
agent --non-interactive "run this task" # No prompts, just output
```

“ *"Perfect for automation, CI/CD pipelines, and batch operations."* ”

Use Cases for One-Shot CLI

Use Case	Example
Code generation	<code>agent "Create a React component for a login form"</code>
Documentation	<code>agent "Generate JSDoc comments for src/api.js"</code>
CI/CD tasks	<code>agent "Review this PR diff for security issues"</code>
Batch processing	Loop through files with <code>agent</code> commands
Pre-commit hooks	<code>agent --mode=ask "Check for console.log statements"</code>

Exercise 5.2 – Steps 1-2

Step 1: Basic one-shot commands:

```
agent "What is the difference between let and const in JavaScript?"  
agent "Write a bash function that checks if a port is in use"  
agent --mode=ask "Explain the git rebase command with examples"
```

Step 2: Specify models:

```
agent --model gpt-5-mini "What does this command do: ls -la | grep .txt"  
agent --model claude-4.5-opus "Design a database schema for a task management system"
```

Exercise 5.2 – Scriptable Code Reviewer

Create `bin/ai-review.sh`:

```
#!/bin/bash
STAGED_FILES=$(git diff --cached --name-only | tr '\n' ', ')

agent --mode=ask "Review these staged files for common issues:
Files: $STAGED_FILES
Check for: debugging statements, unused imports,
security issues, missing error handling. Be concise."
```

Exercise 5.2 — Batch & Git Hooks

Step 4: Batch process files:

```
for file in src/**/*.py; do
  agent --mode=ask --non-interactive \
    "Summarize this Python file in one sentence: $(head -50 $file)"
done
```

Step 5: Pre-commit hook — review staged diff for secrets, debug statements, merge markers

Step 6: CI/CD — analyze test output and suggest fixes for failures

Success Criteria: Ran one-shots • specified models • created reviewer script • understood CI/CD use

Lesson 5.3

Cloud Handoff

Concept · 8 min · Exercise · 10 min

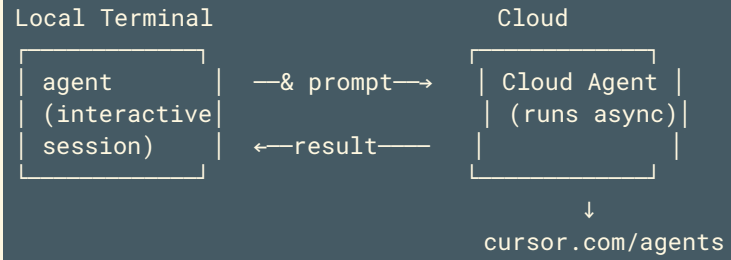
What Is Cloud Handoff?

Send a local conversation to a Cloud Agent:

- Continue from web or mobile (`cursor.com/agents`)
- Let the agent run long tasks while you're away
- Resume the session later from any device

The `&` prefix: Prepend any message with `&` to send it to the cloud.

Cloud Handoff Flow



Exercise 5.3 – Steps 1–3

Step 1: Start local session and hand off:

```
agent  
& "Analyze the entire codebase and create a dependency graph."
```

Step 2: Verify handoff:

```
🚀 Handing off to Cloud Agent...  
✅ Session running at: https://cursor.com/agents/[agent-id]
```

Step 3: Check status via browser or CLI

Exercise 5.3 – Steps 4–6

Step 4: Push existing conversation:

```
& "Continue this conversation in the cloud. I need to log off."
```

Step 5: Long-running task:

```
agent "& Refactor the auth module to use JWT. Update all tests and docs."
```

Step 6: Resume later:

```
agent --resume [agent-id-from-cloud]
```

Cloud Handoff Best Practices

When to Use	When Not to Use
Long-running tasks (>5 min)	Quick questions
When you need to close laptop	Interactive debugging
Overnight batch processing	Tasks needing terminal access
Parallel work streams	Security-sensitive code (local only)

Success Criteria: Sent  message • verified cloud agent • accessed via web

Lesson 5.4

Listing and Resuming Sessions

Concept · 8 min · Exercise · 12 min

Session Management Commands

Command	Purpose
<code>/resume</code>	List all previous sessions and resume one
<code>agent --resume [id]</code>	Resume a specific session by ID
<code>agent --list</code>	List available sessions (alternative)

Tip: Name sessions with the first message:

```
agent "Just say one word: auth-refactor"  
# Session named "auth-refactor Agent"
```

Exercise 5.4 – Steps 1-2

Step 1: Create multiple named sessions:

```
agent "Just say one word: frontend-cleanup" # do work, exit  
agent "Just say one word: db-optimization" # do work, exit  
agent "Just say one word: docs-update"
```

Step 2: List all sessions:

```
/resume  
# 1. frontend-cleanup Agent (2 hours ago)  
# 2. db-optimization Agent (1 hour ago)  
# 3. docs-update Agent (30 minutes ago)
```

Exercise 5.4 – Steps 3–5

Step 3: Resume by ID:

```
agent --resume abc123-def456-ghi789
```

Step 4: Concurrent sessions in different terminals:

```
# Terminal 1: agent --resume frontend-cleanup  
# Terminal 2: agent --resume db-optimization
```

Step 5: Context management:

```
/compress # Summarize conversation, free context window
```

Exercise 5.4 – Steps 6–7 & Best Practices

Step 6: Export session summary as markdown

Step 7: Create `bin/cursor-sessions.sh` to list and manage sessions

Naming: Use `[area]-[task]` format (e.g., `api-auth-fix`)

Context: Use `/compress` on long sessions • cloud handoff for very long tasks

Cleanup: Sessions persist indefinitely – manually complete or discard finished ones

Success Criteria: Created named sessions • listed with `/resume` • resumed • used `/compress`

Module Summary

Lesson	Topic	Key Skill
5.1	Interactive CLI	Real-time terminal AI
5.2	One-Shot CLI	Scripting & automation
5.3	Cloud Handoff	Remote/long-running tasks
5.4	Session Management	Concurrent work handling

Quick Reference Card

BASIC:

```
agent          Start interactive session
agent "prompt"  One-shot command
agent --mode=ask Read-only mode
agent --mode=plan Plan before code
agent --model <name> Specify model
```

IN SESSION: /model /compress /rules /commands /resume /usage

CLOUD: & "message" → cursor.com/agents

KEYS: Shift+Enter (new line) | Ctrl+D twice (exit)

Up Next: Module 6

Custom Prompts and Commands

“ Now that you've mastered terminal-based AI workflows, **Module 6** covers creating reusable prompts, command templates, and team workflows. ”

End of Module 5